

Synera: Synergy Auto-Arena

C++17 · Qt 6 · 单机 PVE 自走棋·高程课程作业

学号：251880336 姓名：李响
已上传 github： https://github.com/winter-warm/slay_the_synera_cpp_homework

目录

- Part 1 ——PA 文档完成度说明
- Part 2 ——Architecture：模块划分与文件职责
 - 2.1 总体分层
 - 2.2 程序入口与 App 流程层
 - 2.3 Core / Entity：棋盘、单位与组件
 - 2.4 Combat：战斗、Buff、Effect 与装备
 - 2.5 World：地图、事件、海克斯与背包
 - 2.6 GUI：页面、棋盘图元与面板
 - 2.7 资源与配置文件
 - 2.8 关键调用链
- Part 3 ——项目亮点
 - 3.1 ECS 组件模式：高解耦的角色架构
 - 3.2 灵活的 Buff / Effect 系统
 - 3.3 仿杀戮尖塔的完整玩法闭环
- Part 4 ——AI 使用说明
- 构建运行

Part 1 ——PA 文档完成度说明

对照 PA 说明文档中的各项需求，完成情况如下：

核心玩法（100% 完成）

PA 需求	完成状态	说明
M × N 六边形棋盘 (8×8)	☑	Board + Hex 实现完整六边形坐标系统与邻居/距离计算
Bench 备战区 (8 格)	☑	支持拖拽布阵、上阵上限校验
Unit 属性: HP / ATK / Range / Max Mana / Mana	☑	StatsComponent 完整实现，另扩展了 Defense / Shield

阵营区分 PlayerCtrl / EnemyCtrl	☑	TeamComponent + teams 枚举
羁绊系统 (traits → 上阵阈值触发)	☑	Bond 枚举含战士/法师/动物/建筑/牧师/魔女/野兽/棋灵/刺客共 9 种
Prep → Combat → Resolve 三个阶段	☑	MapPage → BattlePage (含备战) → 自动战斗 → 结算
自动战斗 (索敌/移动/攻击/技能)	☑	AttackComponent + MoveComponent, 含 BFS 寻路、障碍绕行
GUI 完整展示	☑	Qt6 全套 GUI: 地图页、战斗页、事件页、开始页、HUD、卡牌面板、装备面板

进阶功能 (全部完成)

功能	状态	说明
角色合成升星 (3 合 1)	☑	mergeOwnedCharacterCards(), 同名同星合成
装备系统 (穿戴 + 合成)	☑	基础装备 + 高级装备配方合成
商店系统	☑	5 刷新位 + 购买 + 刷新 + 经验/等级
存档 / 读档	☑	SaveManager + 多槽位
通关记录	☑	RecordManager 记录胜负及阵容
角色回收 (返还金币)	☑	备战区角色回收

额外实现 (超出 PA 范围)

功能	说明
 路线地图	仿杀戮尖塔的节点地图, 含普通战斗/精英/Boss/事件/休息五类节点
 海克斯科技	每层开始可选海克斯卡, 提供长期 Buff / 资源 / 路线修正
 事件系统	JSON 驱动的事件链, 含风险收益选择、嵌套战斗、抽奖等
 Buff / Effect 双轨系统	10 种触发时机 + 灵活组合, 支持数据驱动
 不可选取 / 临时阵营	控制类 Effect, 实现魅惑、潜行等机制
 死亡保护 / 复活	deathProtection, deathRewind 等 Effect
 燃烧 / 再生 / 属性窃取	DOT/HOT tick 类效果
 战斗回放信息	BattleScene 完整动画 + 状态同步

总体完成度: 100% 完成 PA 全部需求, 并在此基础上增加了大量仿杀戮尖塔的扩展玩法。

Part 2 ——Architecture: 模块划分与文件职责

本项目按“入口/流程控制 → 世界推进 → 战斗规则 → 实体组件 → Qt 表现层 → 数据资源”的方向组织。核心思路是:
 UI 不直接承担规则, 规则不直接承担显示, 长期状态统一进 GameState, 战斗中的临时实体由 battlesystem 复制和管理。

2.1 总体分层

src/main.cpp	程序入口
src/app	游戏流程、存档、窗口切换、事件和战斗回流
src/core	Hex 坐标和战术 Board
src/entity	Unit、Character、Object、Obstacle 和 ECS 组件
src/combat	战斗系统、Buff/Effect、装备、战斗配置
src/world	路线地图、事件、海克斯科技、背包/商店状态
src/gui	Qt 页面、战斗场景、HUD、卡牌面板、装备面板
assets	角色、地图、事件、装备、UI 等图片资源

层次	主要职责	上层如何使用
入口层	启动 Qt 应用, 创建主窗口	main.cpp 创建 AppWindow
App 流程层	管理一局游戏的状态变化、 页面跳转、存档读写	GUI 发信号给 GameManager
World 层	生成地图、读取事件、读取 海克斯配置	GameManager 进入节点时调用
Combat 层	载入战斗、复制角色、推进 自动战斗、结算胜负	BattlePage 调用 battlesystem
Entity/Core 层	定义棋盘、坐标、单位、角 色组件	Combat 和 GUI 都依赖它们
GUI 层	页面、场景、拖拽、面板、 动画表现	只展示状态并把用户操作转成信号
数据资源层	JSON 配置和图片素材	Repository / Factory 按需加载

2.2 程序入口与 App 流程层

文件	职责
src/main.cpp	程序入口。创建 QApplication, 设置应用名和版本, 创建 AppWindow 并全屏显示。
src/app/appwindow.h	主窗口声明。持有 GameManager、QStackedWidget、开始页、地图页、事件页、战斗页、卡牌面板、装备面板等对象。
src/app/appwindow.cpp	主窗口实现。负责页面加入 QStackedWidget、连接 Qt 信号槽、在地图/事件/战斗/开始页之间切换, 并处理居中提示 toast。
src/app/gamemanager.h	游戏流程中枢接口。声明新游戏、读档、进节点、选事件、选海克斯、休息、战斗结算、买卡、合成、装备等 slot。
src/app/gamemanager.cpp	游戏流程中枢实现。管理 GameState, 驱动地图推进、事件动作执行、战斗启动/回流、商店刷新、自动保存和通关/失败记录。
src/app/gamestate.h	全局状态结构。保存 seed、玩家生命/金币、地图、已完成节点、当前事件、当前战斗、海克斯、角色卡、装备、商店等。
src/app/gamestate.cpp	GameState 的 JSON 序列化/反序列化。把事件动作、战斗配置、地图、卡牌、装备、商店等状态写入/读出 JSON。
src/app/savemanager.h	存档管理接口。定义保存、读取、删除、查询槽位等操作。
src/app/savemanager.cpp	存档管理实现。把 GameState 写入程序目录下的 save.json, 并从 JSON 恢复游戏。
src/app/recordmanager.h	战绩记录接口。定义通关/失败记录的数据结构和读写方法。
src/app/recordmanager.cpp	战绩记录实现。把一局游戏的结果、用时和阵容等信息记录到本地。
src/app/eventrules.h	事件规则辅助声明。提供事件条件、战斗上下文、奖励/惩罚等规则相关函数接口。

src/app/eventrules.cpp	事件规则辅助实现。处理事件条件判断、节点战斗配置、战斗失败扣血等规则细节。
------------------------	---------------------------------------

App 层的核心调用关系：



2.3 Core / Entity：棋盘、单位与组件

Core 文件

文件	职责
src/core/hex.h	六边形坐标结构。定义 Hex 坐标、合法性、相等/hash 等基础能力，是棋盘寻路和距离计算的坐标基础。
src/core/board.h	战斗棋盘接口。定义格子、区域、占位、移动、邻居、距离、障碍物等操作。
src/core/board.cpp	战斗棋盘实现。读取棋盘 JSON，建立格子表，维护 Unit* 到 Hex 的映射，支持添加/移动/移除单位。
src/core/default_board.json	默认战斗棋盘配置。描述棋盘格子、区域和障碍物。

Entity 基础文件

文件	职责
src/entity/unit.h	所有棋盘单位的抽象基类。保存 id、name、position，并定义纯虚函数 update()。
src/entity/unit.cpp	Unit 基础实现。负责单位 id 自增、位置记录等基础逻辑。
src/entity/character/character.h	战斗角色实体声明。组合 Stats / Attack / Move / Team / Render / Buff 等组件，并保存羁绊、稀有度、装备指针。
src/entity/character/character.cpp	战斗角色实体实现。定义拷贝构造、战斗开始、每帧更新顺序、死亡判断、羁绊查询等逻辑。
src/entity/character/characterfactory.h	角色工厂接口。按 templateId 创建角色，并提供玩家模板信息查询。
src/entity/character/characterfactory.cpp	角色工厂实现。从 template.json 读取角色模板，创建 Character 并装配基础属性、羁绊和默认技能。
src/entity/character/defaultskill.h	默认技能声明。把字符串技能名映射为 Skill 处理函数。
src/entity/character/defaultskill.cpp	默认技能实现。定义角色默认技能、主动技能和 buff 型技能的具体逻辑。
src/entity/character/template.json	角色模板数据。包含角色 id、名称、羁绊、稀有度、基础属性、技能名、队伍等。
src/entity/object/object.h	棋盘对象声明。用于非角色类棋盘单位的基础扩展。
src/entity/object/object.cpp	棋盘对象实现。实现对象类单位的基础行为。
src/entity/obstacle/obstacle.h	障碍物声明。表示棋盘上的阻挡/装饰单位。
src/entity/obstacle/obstacle.cpp	障碍物实现。处理障碍物的渲染资源和棋盘占位行为。

Component 文件

文件	职责
src/entity/component/component.h	组件基类声明。保存 owner 指针和组件可用状态，是所有角色组件的公共父类。
src/entity/component/component.cpp	组件基类实现。提供基础构造、拷贝和启停能力。
src/entity/component/statscomponent.h	属性组件声明。保存 HP、MaxHP、MP、MaxMP、Attack、Defense、Shield、Range。
src/entity/component/statscomponent.cpp	属性组件实现。处理受伤、治疗、防御减伤、护盾抵扣、死亡前 DeathContext 触发。
src/entity/component/attackcomponent.h	攻击组件声明。保存技能列表、攻击冷却、攻击间隔和行动锁。
src/entity/component/attackcomponent.cpp	攻击组件实现。处理普攻、攻击前后 Buff 触发、回蓝、满蓝技能释放、攻击动画请求。
src/entity/component/movecomponent.h	移动组件声明。定义目标选择器、寻路器、移动锁和目标缓存。
src/entity/component/movecomponent.cpp	移动组件实现。提供最近敌人/最高攻击/最低血量目标选择，以及 BFS 寻路和飞行寻路。
src/entity/component/teamcomponent.h	阵营组件声明。保存玩家/敌方等队伍信息。
src/entity/component/teamcomponent.cpp	阵营组件实现。支持读取、修改或临时改变队伍。
src/entity/component/rendercomponent.h	渲染组件声明。保存资源路径和战斗表现请求。
src/entity/component/rendercomponent.cpp	渲染组件实现。发出攻击突进、受伤闪烁、技能爆发等表现层请求。
src/entity/component/buffcomponent.h	Buff 组件声明。把 BuffManager 挂到角色身上，并暴露各种触发时机接口。
src/entity/component/buffcomponent.cpp	Buff 组件实现。转发 onBattleStart、beforeAttack、afterAttack、beforeDeath 等事件到 BuffManager。
src/entity/component/interactcomponent.h	交互组件声明。为可交互单位预留接口。
src/entity/component/interactcomponent.cpp	交互组件实现。处理基础交互行为。
src/entity/component/spritecomponent.h	精灵组件声明。保存精灵图像资源信息。
src/entity/component/spritecomponent.cpp	精灵组件实现。处理精灵资源相关逻辑。
src/entity/equipskill/equipskill.h	装备技能接口。声明装备给角色添加开局 Buff 或主动技能的方法。
src/entity/equipskill/equipskill.cpp	装备技能实现。根据装备定义向角色注入 Buff、技能或属性效果。

2.4 Combat: 战斗、Buff、Effect 与装备

文件	职责
src/combat/battle.json	战斗配置数据。定义不同层、不同节点类型的敌人组合、棋盘 id、属性倍率等。
src/combat/battlerepository.h	战斗配置仓库接口。按地图节点上下文查询对应 BattleConfig。
src/combat/battlerepository.cpp	战斗配置仓库实现。从 battle.json 加载战斗池，按层数和战斗类型选择敌人配置。

src/combat/battlesystem.h	战斗系统接口。声明载入战斗、设置光环、统计羁绊、从备战单位开始战斗、逐帧更新等方法。
src/combat/battlesystem.cpp	战斗系统实现。复制上阵角色、生成敌人、应用装备/羁绊/海克斯、推进自动战斗、移除死亡单位、判断胜负。
src/combat/board_utility.h	棋盘工具接口。提供战斗棋盘相关辅助函数。
src/combat/board_utility.cpp	棋盘工具实现。处理战斗位置、棋盘辅助查询等逻辑。
src/combat/context/combatcontext.h	战斗上下文结构。定义 AttackContext、BeAttackedContext、HealContext、DeathContext 等 Effect 传参对象。

Buff 文件

文件	职责
src/combat/buff/buff.h	Buff 容器声明。保存 Buff 名称、持续时间、冷却、触发次数、触发时机和内部 effect 列表。
src/combat/buff/buff.cpp	Buff 容器实现。负责 apply/remove、触发各类 effect、更新时间、处理到期和移除请求。
src/combat/buff/buffmanager.h	Buff 管理器声明。保存角色身上的 Buff 列表，并声明各触发时机分发函数。
src/combat/buff/buffmanager.cpp	Buff 管理器实现。添加 Buff 时先走 BeforeAddBuff，每帧更新 Buff，到期后移除。
src/combat/buff/bufffactory.h	Buff 工厂接口。按 buffId 和持续时间创建 Buff。
src/combat/buff/bufffactory.cpp	Buff 工厂实现。从 bufflist.json 读取 Buff 配置，并通过 effectfactory 装配 effect。
src/combat/buff/bufflist.json	Buff 数据表。定义 Buff 名称、持续时间、冷却、触发时机、包含哪些 effect。

Effect 文件

文件	职责
src/combat/effect/effect.h	Effect 基类声明。用 std::function 保存 apply/remove/攻击前/受击前/死亡前等 handler。
src/combat/effect/effect.cpp	Effect 基类实现。调用已注入的 handler，并保存 owner、effectId、参数等信息。
src/combat/effect/effectfactory.h	Effect 工厂接口。按 effectId 和参数创建具体 effect 行为。
src/combat/effect/effectfactory.cpp	Effect 工厂实现。根据 effectId 把 effect 绑定到 damage/heal/attribute/control/character 等命名空间的 handler。
src/combat/effect/damage_effect.cpp	伤害类效果实现。例如燃烧、额外伤害、攻击后伤害等。
src/combat/effect/heal_effect.cpp	治疗类效果实现。例如再生、伤害转治疗、治疗加成等。
src/combat/effect/attribute_effect.cpp	属性类效果实现。例如加攻、减伤、免疫、护盾、死亡保护等。
src/combat/effect/control_effect.cpp	控制类效果实现。例如不可选取、临时阵营变化、行动/移动限制等。
src/combat/effect/character_effect.cpp	角色专属效果实现。例如特定角色位移、死亡回溯、专属机制等。
src/combat/effect/effectlist.json	Effect 数据表。定义 effectId、名称和参数。

Equipment 文件

文件	职责
src/combat/equipment/equipment.h	装备数据结构声明。定义基础装备、高级装备、装备组、合成配方等结构。
src/combat/equipment/equipment.cpp	装备基础实现。处理装备结构的辅助逻辑。
src/combat/equipment/equipmentrepository.h	装备仓库接口。查询基础装备、高级装备和合成结果。
src/combat/equipment/equipmentrepository.cpp	装备仓库实现。从 JSON 读取装备定义，支持按 group/id 查找装备和配方。
src/combat/equipment/basic_equipment.json	基础装备配置。定义各基础装备属性和效果。
src/combat/equipment/advanced_equipment.json	高级装备配置。定义高级装备、效果和合成配方。

2.5 World: 地图、事件、海克斯与背包

文件	职责
src/world/map/mapnode.h	地图节点结构。定义节点 id、层数、行列、类型、事件 id、坐标和后继节点。
src/world/map/maplayer.h	地图层声明。保存一层中的节点、行结构、起点/终点，并提供节点查询。
src/world/map/maplayer.cpp	地图层实现。负责按 id 查节点、重建 next 指针等。
src/world/map/mapgraph.h	地图图结构声明。保存多层地图，并提供层查询和 JSON 序列化接口。
src/world/map/mapgraph.cpp	地图图结构实现。处理整张地图的保存/读取和节点类型字符串转换。
src/world/map/mapgenerator.h	地图生成器接口。声明按 seed 生成 MapGraph。
src/world/map/mapgenerator.cpp	地图生成器实现。生成三层路线图，随机节点数和类型，固定特殊事件，并保证相邻行连通。
src/world/event/eventtypes.h	事件与战斗类型定义。定义 EventActionType、BattleConfig、EventOption、EventDefinition 等结构。
src/world/event/eventrepository.h	事件仓库接口。按事件上下文和 stepId 查询事件定义。
src/world/event/eventrepository.cpp	事件仓库实现。从 events.json 读取事件，解析步骤、选项、动作和条件，并做基础校验。
src/world/event/events.json	事件数据表。定义地图事件、休息事件、剧情步骤、选项和动作链。
src/world/hextech/hextech.h	海克斯数据结构声明。定义海克斯卡牌、效果、已选择记录和长期光环。
src/world/hextech/hextech.cpp	海克斯辅助实现。判断海克斯效果类型、处理记录和长期效果。
src/world/hextech/hextechrepository.h	海克斯仓库接口。按层数生成候选卡，按 id 查找定义。
src/world/hextech/hextechrepository.cpp	海克斯仓库实现。从 hextech.json 读取卡牌，按层数和 seed 生成 3 选 1。
src/world/hextech/hextech.json	海克斯配置。定义卡牌标题、描述、图片和长期/即时效果。
src/world/bag/bag.h	背包与商店状态结构。定义拥有的角色卡、装备实例、商店 offer、商店等级/经验等。
src/world/aura/aura.h	光环结构。保存战斗中由海克斯等来源提供的长期效果。

2.6 GUI: 页面、棋盘图元与面板

GUI 层的职责是展示和交互，不直接持久化核心规则。页面收到 GameState 后刷新，用户操作再通过 signal 交给 GameManager 或 BattlePage。

页面文件

文件	职责
src/gui/pages/startpage.h	开始页声明。提供新游戏、读档等信号。
src/gui/pages/startpage.cpp	开始页实现。绘制开始界面，处理新游戏 seed、读取存档等操作。
src/gui/pages/mappage.h	地图页声明。显示路线地图、顶部状态、背包/商店/装备入口和节点选择信号。
src/gui/pages/mappage.cpp	地图页实现。根据 MapGraph 绘制节点和连线，处理节点点击、保存、返回开始页等。
src/gui/pages/eventpage.h	事件页声明。展示事件文本、选项、海克斯选择、休息选择、招募/装备选择等 UI。
src/gui/pages/eventpage.cpp	事件页实现。根据 CurrentEventState 切换不同选择模式，并发出对应事件信号。
src/gui/pages/battlepage.h	战斗页声明。管理备战、开始战斗、战斗结束、宝箱奖励、面板入口等交互。
src/gui/pages/battlepage.cpp	战斗页实现。连接 BattleScene 和 battlesystem，收集上阵摆放，驱动战斗计时与结果显示。

战斗场景与棋盘图元

文件	职责
src/gui/battle/battlescene.h	战斗场景声明。管理棋盘格、单位图元、备战区、拖拽、障碍物、动画和宝箱。
src/gui/battle/battlescene.cpp	战斗场景实现。把 Board 和 Character 状态同步成 Qt 图元，处理拖拽布阵和战斗动画。
src/gui/board/hexlayout.h	六边形布局声明。负责六边形坐标和屏幕坐标之间的转换。
src/gui/board/hexlayout.cpp	六边形布局实现。计算 hex 中心点、顶点、多边形和命中区域。
src/gui/board/griditem.h	棋盘格图元声明。显示单个 hex 格子的区域、悬停和可放置状态。
src/gui/board/griditem.cpp	棋盘格图元实现。绘制六边形格子并响应状态变化。
src/gui/board/unititem.h	单位图元声明。显示战斗单位头像/棋子、血条、蓝条和动画请求。
src/gui/board/unititem.cpp	单位图元实现。处理单位位置、受击闪烁、攻击突进、技能爆发等表现。
src/gui/board/benchslotitem.h	备战区格子声明。表示一个可放置/拖拽的备战槽位。
src/gui/board/benchslotitem.cpp	备战区格子实现。处理备战角色显示、拖拽和选中效果。
src/gui/board/propitem.h	道具/场景物图元声明。用于显示障碍物、宝箱等棋盘元素。
src/gui/board/propitem.cpp	道具/场景物图元实现。绘制图片资源并同步位置。

HUD 与面板

文件	职责
src/gui/HUD/characterhud.h	角色 HUD 声明。显示角色属性、技能或状态信息。
src/gui/HUD/characterhud.cpp	角色 HUD 实现。根据角色数据刷新 HUD。
src/gui/widgets/gamehud.h	游戏 HUD 声明。显示生命、金币、层数、用时等全局信息。
src/gui/widgets/gamehud.cpp	游戏 HUD 实现。把 GameState 中的核心数值同步到界面。
src/gui/widgets/cardpanel.h	卡牌/商店面板声明。展示拥有角色卡、商店 offer、合成/购买/训练/回收等操作。
src/gui/widgets/cardpanel.cpp	卡牌/商店面板实现。处理卡牌列表、商店购买、升星合成、训练选择和面板动画。
src/gui/widgets/equipmentpanel.h	装备面板声明。展示装备背包、穿戴、卸下、合成等操作。
src/gui/widgets/equipmentpanel.cpp	装备面板实现。处理装备实例、合成配方、装备到角色卡、战斗准备模式下的装备选择。

2.7 资源与配置文件

路径	职责
assets/characters/	角色卡面和棋子图。角色模板中的资源路径会指向这里。
assets/events/	事件背景图。事件 JSON 的 backgroundPath 使用这些图。
assets/maps/	路线地图背景。每一层地图有对应背景。
assets/nodes/	地图节点图标。普通战斗、精英、Boss、事件、休息等节点使用不同图标。
assets/equipment/	装备图标。装备面板和奖励展示使用。
assets/obstacles/	战斗棋盘障碍物图。棋盘 JSON 会引用这些资源。
assets/pages/	开始页等页面背景。
assets/ui/	按钮、标题、HUD、宝箱等 UI 图片资源。
CMakeLists.txt	构建脚本。收集 src 下 C++ 文件，链接 Qt，并在构建后复制 JSON 和 assets 到运行目录。

运行时配置复制关系：

```
src/core/default_board.json      -> build/boards/default_board.json
src/world/event/events.json      -> build/events/events.json
src/combat/battle.json           -> build/battle/battle.json
src/combat/equipment/*.json      -> build/equipment/*.json
src/world/hextech/hextech.json   -> build/hextech/hextech.json
assets/                          -> build/assets/
```

2.8 关键调用链

新游戏到地图

```
main.cpp
-> AppWindow
-> StartPage::newGameRequested
-> GameManager::newGame(seed)
-> MapGenerator::generate(seed)
-> GameManager::refreshShopOffers()
-> GameManager::unlockLayerStart(1)
-> AppWindow 切换到 MapPage
```

点击地图节点

```
MapPage::nodeSelected(nodeId)
-> GameManager::enterMapNode(nodeId)
-> 根据 event_id 分流:
    Start      -> loadHexTechEvent()
    Rest       -> loadRestEvent()
    Normal/Elite -> BattleRepository::battleFor() -> startBattle()
    Boss       -> BattleRepository::battleFor() -> startBattle()
    Event      -> EventRepository::eventFor() -> EventPage
```

一场战斗

```

BattlePage 收集备战区 placements
-> GameManager::startPreparedBattle(placements)
-> battlesystem::startFromPreparedUnits()
    复制玩家角色和敌人
    应用装备 Buff
    应用羁绊加成
    应用海克斯光环
    放入 Board
-> battlesystem::update(deltaTime)
    Character::update()
        BuffComponent::update()
        MoveComponent::update()
        AttackComponent::update()
    removeDeadCharacters()
    finishIfTeamDead()
-> BattlePage::battleFinished
-> GameManager::finishBattle()

```

事件动作

```

EventPage::eventOptionSelected(index)
-> GameManager::chooseEventOption(index)
-> GameManager::executeEventActions()
-> GameManager::executeEventAction()
    FinishNode / GoToStep / StartBattle / GrantGold / LoseHp / ChooseRecruit ...

```

Part 3 ——项目亮点

3.1 ECS 组件模式：高解耦的角色架构

传统做法通常把所有属性塞进 Character 类，导致类膨胀且难以扩展。本项目采用了 ECS (Entity-Component) 组件化设计：

Character (Entity)	
└ StatsComponent	—— 属性：HP/ATK/MP/Defense/Shield/Range
└ AttackComponent	—— 战斗：普攻冷却/技能列表/行动锁
└ MoveComponent	—— 移动：寻路/障碍绕行/路径缓存
└ TeamComponent	—— 阵营：玩家/敌方/临时转换
└ RenderComponent	—— 渲染：棋子图标/卡面/动画帧
└ BuffComponent	—— Buff：挂载/触发/到期移除
└ Equipment*	—— 装备槽（最多 1 件）

设计优势：

- **单一职责**：每个 Component 只管理一个维度，互不干扰。改移动逻辑不需要看战斗代码。
- **可拔插**：新角色只需要组合不同 Component，不需要修改基类。Obstacle（障碍物）复用 Unit 体系，但禁用了不需要的组件。

- **可扩展**: 添加新系统 (如飞行、隐身) 只需新增一个 Component 或在现有组件上加一个 lock 计数器, 不影响已有逻辑。
- **复制安全**: 每个 Component 都提供了拷贝构造函数 (Component(const Component&, Character*)), 合成升星时能正确复制所有状态。

3.2 灵活的 Buff / Effect 系统

这是整个战斗系统的核心, 设计核心是 **Buff 作为容器** + **Effect 作为行为** 的双层解耦:

Buff 层 (容器)

```
class buff {
    QString name;
    vector<unique_ptr<effect>> effects; // 一个 Buff 可包含多个 Effect
    float duration;                    // 持续时间 (-1 = 永久)
    float cooldown;                    // 冷却时间
    BuffTrigger trigger;               // 触发时机
    int maxtrigger;                    // 最大触发次数
    short priority;                    // 优先级
};
```

10 种触发时机覆盖完整战斗生命周期:

触发时机	典型用途
OnBattleStart	战斗开始加属性、飞行、视野
BeforeAttack	攻击增伤、暴击
AfterAttack	攻击后附加伤害、女巫化
BeforeBeAttacked	伤害减免、闪避、反伤
AfterBeAttacked	受伤后加攻、位移、模仿
BeforeHeal	治疗加成
AfterSkill	技能后效果
BeforeAddBuff	Buff 抗性/免疫
BeforeDeath	死亡保护、复活
OnTurnStart	DOT 燃烧、HOT 再生

Effect 层 (行为)

```
class effect {
    // 每种触发时机对应一个 std::function 回调
    AttackHandler beforeAttackFunc; // 可通过 setBeforeAttack() 注入
    BeAttackedHandler beforeBeAttackedFunc;
    TurnHandler onTurnStartFunc;
    // ... 共 10 种 handler
};
```

关键设计巧思:

1. **函数注入而非继承**——Effect 不使用虚函数多态, 而是通过 std::function 注入行为。effectfactory 从 JSON 读取配置后, 按 effectId 装配对应的 handler。这意味着添加新效果只需在 factory 的 switch 里加一个 case, 不需要新建子类。

1. **namespace 分组管理**——20+ 种 Effect 按类型分到 5 个 namespace:
- damage_effect ——女巫化、燃烧伤害

• heal_effect ——反转伤害为治疗、再生

• attribute_effect ——属性修改、伤害倍率、减伤、免疫、死亡保护

• control_effect ——临时阵营（魅惑）、不可选取（潜行）

• character_effect ——角色专属：Coco 位移、Noah 减伤再生、Levitation 飞行、死亡回溯等
1. **Context 传参模式**——每个触发时机有独立 Context 结构体(AttackContext, BeAttackedContext, DeathContext 等), Effect 可以修改其中的 cancelled、damage 等字段来拦截/修改后续行为。例如 damageImmunity 通过设置 cancelled = true 完全阻止伤害。
1. **JSON 数据驱动**——Buff 列表 (bufflist.json) 和 Effect 列表 (effectlist.json) 完全由数据定义，策划可以直接修改 JSON 来调整效果参数，无需重新编译。
1. **BuffManager 优先级排序**——多个 Buff 按优先级排序触发，确保关键效果（如死亡保护）在其他效果之前执行。

3.3 仿杀戮尖塔的完整玩法闭环

在 PA 要求的基础自走棋之上，补充了类杀戮尖塔的完整 Roguelike 爬塔体验：

开始 → 选择海克斯 → 推进地图节点 → 战斗/事件/休息 → Boss 层 → 下一层 → 通关/失败

系统	实现细节
路线地图	MapGraph 生成分层节点图，玩家逐节点推进，选择路线
海克斯科技	每层开始 3 选 1，效果涵盖长期 Buff、资源奖励、路线/掉落修正
事件系统	JSON 驱动的事件链,支持多选项、条件判定、嵌套战斗、抽奖(LotteryScratch)等 18 种 Action
休息节点	休息回血 / 训练升星 / 挖宝得装备 / 锻造合成，4 种选择
Boss 战	三层 Boss，击败即通关
存档系统	手动 + 自动存档，多槽位
通关记录	记录胜负、阵容、用时

Part 4 ——AI 使用说明

本项目在开发过程中使用了 AI（Claude Code 和 codex）辅助，以下是各模块的 AI 参与程度说明：

4.1 Entity 部分——主要自己完成，AI 修 Bug 和机械化劳动

模块	文件
Unit 基类	entity/unit.h/cpp
Character	entity/character/character.h/cpp
各 Component	entity/component/*.h/cpp
CharacterFactory	entity/character/characterfactory.h/cpp
DefaultSkill	entity/character/defaultskill.h/cpp

Object / Obstacle	entity/object/, entity/obstacle/
EquipSkill	entity/equipskill/

4.2 图片素材——完全 AI 生成

资源类型	路径	生成方式
角色卡面 + 棋子图	assets/characters/*/card.png, pawn.png	AI 图像生成
事件背景图	assets/events/	AI 图像生成
地图背景	assets/maps/	AI 图像生成
节点图标	assets/nodes/	AI 图像生成
装备图标	assets/equipment/	AI 图像生成
障碍物图	assets/obstacles/	AI 图像生成
UI 元素	assets/ui/, assets/pages/	AI 图像生成

提示词在 [github](#) 说放了

4.3 GUI 部分——大部分 AI 生成

模块	文件	AI 参与度	说明
StartPage	gui/pages/startpage.h/cpp	AI 生成, 不太懂 Qt	
MapPage	gui/pages/mappage.h/cpp	AI 生成	
BattlePage	gui/pages/battlepage.h/cpp	在原有上改的	
EventPage	gui/pages/eventpage.h/cpp	自己写的	
BattleScene	gui/battle/battlescene.h/cpp	战斗棋盘 Qt 场景, AI 生成	
Board UI	gui/board/hexlayout.h/cpp, unititem, griditem, benchslotitem, propitem	六边形的坐标表示是在 b 站学的, 没有用原来的坐标	
HUD / Widgets	gui/HUD/, gui/widgets/	CardPanel, Equipment-Panel, GameHUD 等	

由于对 Qt 的 *Graphics View / Widget* 体系不太熟悉, GUI 层绝大部分代码由 AI 生成, 自己主要负责接口定义和决定采用的数学方法, 并给出优化方案

4.4 Combat 部分——AI 辅助, 自己搭框架

模块	自己完成	AI 完成
BattleSystem	.h 头文件 (接口设计 + 架构)	.cpp 实现文件

Effect 基类	设计 Effect 抽象基类 + 10 种 handler 函数指针注入模式	—
Effect 派生逻辑	搭建 namespace 分组 + factory 框架	各具体 Effect handler 实现 (20+ 种)
Buff	设计 Buff 类 + BuffManager 容器 + 10 种触发时机枚举	—
BattleRepository	接口设计	.cpp JSON 加载实现
Board Utility	自己完成	—
Equipment	接口设计	.cpp JSON 加载
Context	combatcontext.h 全部 Context 结构体设计	—

Combat 模块的核心架构 (Effect 基类、Buff 容器、Context 传参、Factory 装配) 全部自己设计。由于 Effect 种类特别多 (20+ 种), 具体的 handler 实现交给 AI 批量生成; BattleSystem 采用”自己写 .h 定义接口, AI 写 .cpp 实现细节”的协作模式。

4.5 其余部分

模块	AI 参与度	说明
App 层 (gamemanager, gamestate, savemanager, recordmanager, eventrules)	游戏流程控制基本由 AI 完成, 这个太底层了我做不来, 没法像 ai 一样记住那么多模块的关系	
World 层 (mapgenerator, mapgraph, maplayer, eventrepository, hextechrepository)	ai 太笨了, 看不懂需要绘图的 map(主要是它没办法运行) 最后变成了我写	
Core 层 (board, hex)	原有, 修改了一下	

资源结构

assets/characters	角色卡面和棋子图
assets/events	事件背景图
assets/maps	路线地图背景
assets/nodes	地图节点图标
assets/equipment	装备图标
assets/obstacles	棋盘障碍物图
assets/pages	开始页背景
assets/ui	按钮、标题、HUD、宝箱等 UI 图

构建运行

环境要求

- **Qt 6** (Core、Gui、Widgets、Svg、Network 模块)
- **CMake** 3.16+
- **编译器**: 支持 C++17 (GCC 13+ / MSVC 2022+ / MinGW-w64)

构建

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH="/path/to/Qt/6.x.x/mingw_64"  
cmake --build build --config Release
```